

Условная компиляция

```
fn greeting() -> &'static str {  
    #[cfg(feature = "verbose")]  
    {  
        #[cfg(target_os = "linux")] { return "linux"; }  
        #[cfg(target_os = "windows")] { return "windows"; }  
        #[cfg(not(any(target_os = "linux", target_os = "windows")))]  
        { return "unknown"; }  
    }  
    #[cfg(not(feature = "verbose"))] { return ""; }  
}  
  
fn main() {  
    println!("hello {}", greeting());  
}  
  
// cargo run --features verbose # -> hello linux  
// cargo run # -> hello
```

Классическое использование трейтов

```
trait Animal { fn speak(&self); }

struct Dog;
struct Cat;

impl Animal for Dog { fn speak(&self) { println!("Woof!"); } }
impl Animal for Cat { fn speak(&self) { println!("Meow!"); } }

// Will be generated 2 functions
fn announce<T: Animal>(animal: T) { animal.speak(); }

fn main() {
    announce(Dog); // Woof
    announce(Cat); // Meow
}
```

Реализация по умолчанию

```
trait Animal { fn speak(&self); }

impl<T> Animal for T {
    fn speak(&self) { println!("Hello"); }
}

struct Cat;

fn announce<T: Animal>(animal: T) {
    animal.speak();
}

fn main() {
    announce(Cat); // Hello
}
```

```
trait Animal { fn speak(&self); }
impl<T> Animal for T {
    fn speak(&self) { println!("Hello"); }
}
struct Cat;
impl Animal for Cat { fn speak(&self)
    { println!("Meow!"); } }
fn announce<T: Animal>(animal: T)
    { animal.speak(); }
fn main() {
    announce(Cat);
}
// 3 | impl<T> Animal for T {
//    | first implementation here
// 10 | impl Animal for Cat { fn speak(&self)
//    | conflicting implementation for `Cat`
```

feature(specialization) — nightly версия

```
// rustup override set nightly
#![feature(specialization)]
trait Animal {
    fn speak(&self);
}
impl<T> Animal for T {
    default fn speak(&self) {
        println!("Hello");
    }
}
struct Cat;
struct Dog;
```

```
impl Animal for Cat {
    fn speak(&self) {
        println!("Meow!");
    }
}
fn announce<T: Animal>(animal: T) {
    animal.speak();
}
fn main() {
    announce(Cat); // Meow!
    announce(Dog); // Hello
}
```

Ассоциированные типы

```
trait Container {  
    type Item;  
    fn get(&self) -> Option<&Self::Item>;  
}  
  
struct VecWrapper<T> {  
    data: Vec<T>,  
}  
  
impl<T> Container for VecWrapper<T> {  
    type Item = T;  
    fn get(&self) -> Option<&Self::Item> {  
        self.data.first()  
    }  
}  
  
fn main() {}
```

```
#include <concepts>  
  
template<typename T>  
class VecWrapper {  
public:  
    using Item = T;  
    Item* get() { return nullptr; }  
};  
  
template<typename C>  
concept Container = requires(C c) {  
    typename C::Item;  
    { c.get() } ->  
        std::same_as<typename C::Item*>;  
};  
  
static_assert(Container<VecWrapper<int>>);  
  
int main() {}
```

CRTP в Rust

```
trait Base<T: Base<T>> {  
    fn interface(&self) {  
        self.impl_method();  
    }  
    fn impl_method(&self);  
}  
  
struct Derived;  
impl Base<Derived> for Derived {  
    fn impl_method(&self) {  
        println!("Derived implementation");  
    }  
}
```

Маркерные трейты

```
use std::rc::Rc;

// Send - threadsafety compiler marker
fn is_send<T: Send>() {}

fn main() {
    is_send::<i32>();           // OK
    is_send::<String>();        // OK
    // is_send::<Rc<i32>>();     // Error: Rc is not Send
}
```

Несколько трейт-bounds и where-клаузы

```
use std::fmt::Debug;

fn print_if_eq<T: PartialEq + Debug>(
    a: T, b: T
) {
    if a == b { println!("{:?}", a); }
}

fn main() {
    print_if_eq(42, 42);           // 42
    print_if_eq("hi", "bye");     // (nothing)
}
```

```
use std::fmt::{Debug, Display};

fn log<T, U>(val: T, label: U)
where
    T: Debug + Clone + Send,
    U: Display,
{
    println!("{}: {:?}", label, val);
}

fn main() {
    log(vec![1, 2, 3], "numbers");
    // numbers: [1, 2, 3]
}
```


impl Trait

```
use std::fmt::Display;

fn print_val(val: impl Display) {
    println!("{}", val);
}

fn print_val2<T: Display>(val: T) {
    println!("{}", val);
}

fn main() {
    print_val(42);
    print_val("hello");
}
```

```
fn make_adder(x: i32) -> impl Fn(i32) -> i32 {
    move |y| x + y
}

fn make_adder_dyn(x: i32) ->
    Box<dyn Fn(i32) -> i32> {
    Box::new(move |y| x + y)
}

fn main() {
    let add5 = make_adder(5);
    println!("{}", add5(3)); // 8
}
```